| | | | |
|---|---|---|---|
| **11** | Link Jira tasks with Confluence to streamline task tracking and progress monitoring for the Library Management System development.<br>• Create a new page in Confluence titled "Library Management System Project Overview."<br>• Embed at least 5 Jira issues related to the development of the Library Management System (e.g., tasks from the sprint like "Develop book search functionality," "Create user login page," etc.).<br>• Use the Jira macro to display issues with status (e.g., "To Do," "In Progress," "Done").<br>• Add a progress bar in the Confluence page to visually track the completion of each embedded Jira task (e.g., percentage of tasks completed in the sprint).<br>• Submit a screenshot of the Confluence page showing the embedded Jira tasks and the progress bar. | Jira & Confluence | CO2 |

**Objective**
To link Jira issues with Confluence pages in order to streamline task tracking, monitor progress, and visualize sprint completion for the Library Management System development.

**Tools Required**
- Jira Software
- Confluence

(Both should belong to the same Atlassian workspace.)

**Step-by-Step Procedure**

- **Step 1: Create Jira Issues**
  1. Open **Jira**
  2. Go to your project → Example:
     *Library Management System Project*
  3. Create at least **5 issues**:

| Issue Type | Issue Name |
|---|---|
| Story | Develop book search functionality |
| Task | Create user login page |
| Task | Implement book reservation |
| Task | Generate monthly report |
| Bug / Task | Email overdue notification |

  4. Assign different statuses:
- To Do
- In Progress
- Done

- **Step 2: Open Confluence**
  1. Open **Confluence**
  2. Go to your project space.

- **Step 3: Create a New Page**
  1. Click **Create**
  2. Title the page:

**Library Management System Project Overview**

  3. Click **Create Page**

- **Step 4: Add Project Introduction**

Type:
This page tracks Jira tasks and progress for the Library Management System development project.

- **Step 5: Insert Jira Macro for Issues**
  1. Click **+ (Insert) → Jira Issue / Jira Macro**
  2. Choose **Search for issues**
  3. Select your Jira project
  4. Filter issues for the sprint
  5. Click **Insert**

This embeds all Jira issues.

- **Step 6: Display Status Column**

In macro settings:

✓ Enable columns:

- Issue Key
- Summary
- Status
- Assignee

Now tasks show **To Do, In Progress, Done**.

- **Step 7: Add Progress Bar**
  1. Click **+ → Insert → Jira Roadmap / Status Macro / Progress Macro**
  2. Or manually calculate:

Example:
Total Tasks: 5
Completed: 2
Progress: 40%

  3. Insert **Progress Bar Macro**
  4. Set value = 40%

- **Step 8: Organize Page Layout**

Use headings:
## Sprint Tasks
## Task Status
## Sprint Progress
Place:

- Jira issue table under Sprint Tasks
- Progress bar under Sprint Progress

- **Step 9: Save Page**

Click **Publish**

- **Step 10: Take Screenshot**

Take screenshot showing:

✓ Page title
✓ Embedded Jira issues
✓ Status column
✓ Progress bar

**Screenshot Caption for Record**
**Confluence page showing embedded Jira issues and sprint progress for Library Management System project.**

**Result**
Thus, Jira tasks were successfully linked with Confluence to track development progress of the Library Management System using embedded Jira issues and a progress bar.

| | | | |
|---|---|---|---|
| **12** | You are designing a Task Management System for a small team. The system should include the following features:<br><br>1. User Login and Role Assignment<br>2. Task Creation and Assignment<br>3. Task Prioritization and Deadlines<br>4. Progress Tracking and Reporting<br><br>Prioritize these requirements using the MoSCoW and Kano models in Jira. | Jira | CO2 |

**Aim**

To prioritize Task Management System requirements using **MoSCoW** and **Kano** models in Jira.

**System Requirements**

1. User Login and Role Assignment
2. Task Creation and Assignment
3. Task Prioritization and Deadlines
4. Progress Tracking and Reporting

**PART A — MoSCoW Prioritization in Jira**

**Step 1: Create Jira Project**

1. Login to Jira
2. Click **Projects → Create project**
3. Select **Scrum Software Development**
4. Choose **Team-managed project**
5. Project name: Task Management System
6. Click **Create**

**Step 2: Create Backlog Items**

Create 4 **Story** issues:

| Story Name |
|---|
| User Login and Role Assignment |
| Task Creation and Assignment |
| Task Prioritization and Deadlines |
| Progress Tracking and Reporting |

**Step 3: Create Custom Field for MoSCoW**

1. Click Settings → Issues
2. Click **Custom fields → Create custom field**
3. Select **Single Select List**
4. Name it: MoSCoW Priority
5. Add options:
    - Must Have
    - Should Have
    - Could Have
    - Won't Have
6. Associate with Scrum screens.

**Step 4: Assign MoSCoW Priority**

Open each story and assign:

| Requirement | MoSCoW |
|---|---|
| User Login and Role Assignment | Must Have |
| Task Creation and Assignment | Must Have |
| Task Prioritization and Deadlines | Should Have |
| Progress Tracking and Reporting | Could Have |

**PART B — Kano Model Prioritization in Jira**

**Step 5: Create Custom Field for Kano Model**

1. Go to **Custom fields → Create**

2. Choose **Single Select List**
3. Name: Kano Category
4. Options:
    - Basic (Must-be)
    - Performance
    - Excitement
    - Indifferent

**Step 6: Assign Kano Categories**

| Requirement | Kano Category |
|---|---|
| User Login and Role Assignment | Basic |
| Task Creation and Assignment | Basic |
| Task Prioritization and Deadlines | Performance |
| Progress Tracking and Reporting | Excitement |

**PART C — Display Prioritization in Backlog**
**Step 7: Configure Columns**

1. Open Backlog
2. Click **View settings → Columns**
3. Add:
    - MoSCoW Priority
    - Kano Category

Now backlog shows prioritization clearly.

**Screenshots to Capture**

1. Backlog with MoSCoW & Kano columns
2. Issue detail showing MoSCoW + Kano values

**Observation Table (for Lab Record)**

| Feature | MoSCoW | Kano |
|---|---|---|
| User Login and Role Assignment | Must Have | Basic |
| Task Creation and Assignment | Must Have | Basic |
| Task Prioritization and Deadlines | Should Have | Performance |
| Progress Tracking and Reporting | Could Have | Excitement |

**Result**

The requirements of the Task Management System were successfully prioritized using MoSCoW and Kano models in Jira.

| | | | |
|---|---|---|---|
| **13** | You are tasked with developing an Online Learning Platform. The platform should include the following functionalities:<br><br>1. Course Enrollment and Registration<br>2. Video Lecture Streaming<br>3. Interactive Quizzes and Assignments<br>4. Progress Tracking Dashboard<br>5. Peer-to-Peer Discussion Forums<br>6. Certificate Generation<br><br>Use Jira to categorize and prioritize these requirements using MoSCoW and Kano techniques. | Jira | CO2 |

**Aim**

To categorize and prioritize Online Learning Platform requirements using **MoSCoW** and **Kano techniques** in Jira.

**Requirements List**

1. Course Enrollment and Registration
2. Video Lecture Streaming
3. Interactive Quizzes and Assignments
4. Progress Tracking Dashboard
5. Peer-to-Peer Discussion Forums

6. Certificate Generation

**PART A — Project Setup in Jira**

**Step 1: Create Jira Project**

1. Login to Jira.
2. Click **Projects → Create Project**.
3. Choose **Scrum Software Development** template.
4. Select **Team-managed project**.
5. Enter:
    - Project Name: Online Learning Platform
    - Key: OLP
6. Click **Create Project**.

**PART B — Add Requirements as Backlog Items**

**Step 2: Create Stories**

Create 6 **Story** issues with these titles:

| Story Name |
|---|
| Course Enrollment and Registration |
| Video Lecture Streaming |
| Interactive Quizzes and Assignments |
| Progress Tracking Dashboard |
| Peer-to-Peer Discussion Forums |
| Certificate Generation |

**PART C — MoSCoW Prioritization**

**Step 3: Create MoSCoW Custom Field**

1. Click **Settings → Issues → Custom fields**
2. Click **Create custom field**
3. Select **Single Select List**
4. Name it: MoSCoW Priority
5. Add values:
    - Must Have
    - Should Have
    - Could Have
    - Won't Have
6. Associate it with Scrum screens.

**Step 4: Assign MoSCoW Values**

| Requirement | MoSCoW |
|---|---|
| Course Enrollment and Registration | Must Have |
| Video Lecture Streaming | Must Have |
| Interactive Quizzes and Assignments | Should Have |
| Progress Tracking Dashboard | Should Have |
| Peer-to-Peer Discussion Forums | Could Have |
| Certificate Generation | Could Have |

**PART D — Kano Model Categorization**

**Step 5: Create Kano Custom Field**

1. Go to **Custom fields → Create**
2. Select **Single Select List**
3. Name it: Kano Category
4. Add options:
    - Basic (Must-Be)
    - Performance
    - Excitement
    - Indifferent

**Step 6: Assign Kano Categories**

| Requirement | Kano |
|---|---|
| Course Enrollment and Registration | Basic |
| Video Lecture Streaming | Basic |
| Interactive Quizzes and Assignments | Performance |
| Progress Tracking Dashboard | Performance |
| Peer-to-Peer Discussion Forums | Excitement |
| Certificate Generation | Excitement |

**PART E — Display in Backlog**

**Step 7: Show Custom Fields in Backlog**

1. Open **Backlog**
2. Click **View Settings → Columns**
3. Add:
    - MoSCoW Priority
    - Kano Category

Now backlog shows both prioritization models.

**Screenshots to Capture**

For lab submission:

1. Backlog showing MoSCoW & Kano columns
2. Issue detail page with MoSCoW and Kano values

**Observation Table**

| Feature | MoSCoW | Kano |
|---|---|---|
| Course Enrollment and Registration | Must | Basic |
| Video Lecture Streaming | Must | Basic |
| Interactive Quizzes and Assignments | Should | Performance |
| Progress Tracking Dashboard | Should | Performance |
| Peer-to-Peer Discussion Forums | Could | Excitement |
| Certificate Generation | Could | Excitement |

**Result**

The Online Learning Platform requirements were successfully categorized and prioritized using MoSCoW and Kano techniques in Jira.

| | | | |
|---|---|---|---|
| **14** | Demonstrate how to work collaboratively in Git/GitHub on a project using the fork-and-pull request workflow.<br><br>Tasks:<br>1. Fork an existing public GitHub repository (e.g., a sample JavaScript or Python project).<br>2. Clone the forked repository to your local machine using Git.<br>3. Create a new branch for the feature or change you want to work on.<br>4. Make modifications or add new features (e.g., add a function, fix a bug, or update the README).<br>5. Commit your changes and push the branch to GitHub.<br>6. Go to the GitHub repository and create a pull request to merge your feature branch into the main branch.<br>7. Review the pull request and provide feedback on the changes. Respond to feedback by making additional commits to the feature branch if necessary.<br>9. Once the pull request is approved, merge it into the main branch.<br>8. Finally submit the link to the pull request along with a summary of the changes you made and how you collaborated. | Git/GitHub | CO3 |

**Aim**

To demonstrate collaborative software development using Git and GitHub through fork, branch, pull request, review, and merge workflow.

**Requirements**

- Git installed
- GitHub account
- Internet connection

**Step 1: Fork a Public Repository**

1. Go to GitHub.
2. Search for a public repository (example: python-sample-project).
3. Open the repository.
4. Click **Fork** (top-right corner).
5. Select your GitHub account.

Repository is now copied into your account.

**Step 2: Clone the Forked Repository**

1. Open your forked repository.
2. Click **Code → HTTPS → Copy URL**.
3. Open Git Bash / Terminal.
4. Run:

git clone https://github.com/your-username/repository-name.git

5. Move into project folder:

cd repository-name

**Step 3: Create a New Feature Branch**

git checkout -b feature-readme-update

**Step 4: Make Changes**

Example: Update README.md

Add:

This project demonstrates collaborative GitHub workflow using pull requests.

Or add a new function in Python/JS file.

Save file.

**Step 5: Commit the Changes**

```
git status
git add .
git commit -m "Updated README with collaboration workflow description"
```

**Step 6: Push Branch to GitHub**

```
git push origin feature-readme-update
```

**Step 7: Create Pull Request**

1. Go to your forked repository on GitHub.
2. Click **Compare & pull request**.
3. Set:
    - Base repository: Original repo → main
    - Head repository: Your fork → feature-readme-update
4. Add title:
5. Updated README for GitHub collaboration workflow
6. Add description.
7. Click **Create pull request**.

**Step 8: Review and Feedback**

1. Open Pull Request.
2. Click **Files changed**.
3. Add comment like:
4. The README improvement clearly explains the collaboration process.
5. Submit review.

**Step 9: Respond to Feedback**

If changes are requested:

1. Edit file again locally.
2. Commit:

```
git add .
git commit -m "Improved README based on review feedback"
```

3. Push:

```
git push origin feature-readme-update
```

Pull request updates automatically.

**Step 10: Merge Pull Request**

Once approved:

1. Click **Merge pull request**
2. Click **Confirm merge**

Changes merged into main branch.

**Step 11: Final Submission**

Submit:

- Pull Request Link
- Summary of changes
- Collaboration description

**Sample Summary for Lab Record**

**Pull Request Link:**
https://github.com/original-user/repo/pull/1

**Summary of Changes:**
Updated the README file to include a description of GitHub collaboration workflow using fork and pull request method.

**Collaboration Process:**
The feature branch was created, changes were committed, pull request was submitted, reviewed, feedback was addressed, and finally merged into the main branch.

**Result**

Thus, collaborative development using GitHub fork and pull request workflow was successfully implemented.

| | | | |
|---|---|---|---|
| **15** | Demonstrate how to work with Git branches and resolve merge conflicts when collaborating with others.<br><br>Tasks:<br>1. Clone a shared repository to your local machine.<br>2. Create a new branch and switch to it.<br>3. Make changes to a file (e.g., update a README or modify code in a specific function).<br>4. Commit your changes.<br>5. Push the changes to the remote repository.<br>6. Before merging, pull the latest changes from the main branch.<br>7. Switch back to your feature branch.<br>8. Merge the main branch into your feature branch.<br>9. If there are conflicts, resolve them manually by editing the conflicting files. After resolving conflicts, mark the conflicts as resolved.<br>10. Commit the resolved merge.<br>11. Push your feature branch with the merged changes to GitHub and create a pull request.<br>12. Submit a summary of the steps you performed, the conflicts you encountered, and how you resolved them. | Git/GitHub | CO3 |

**Aim**

To demonstrate branching, merging, and conflict resolution in Git during collaborative development.

**Requirements**

- Git installed
- GitHub account
- Shared repository access

**Step 1: Clone Shared Repository**

git clone https://github.com/team/repository-name.git
cd repository-name

**Step 2: Create and Switch to Feature Branch**

git checkout -b feature-update-readme

**Step 3: Make Changes**

Open README.md and modify a section:
This project demonstrates Git branching and conflict resolution.
Save file.

**Step 4: Commit Your Changes**

git add README.md
git commit -m "Updated README with branching information"

**Step 5: Push Feature Branch**
git push origin feature-update-readme

**Step 6: Pull Latest Main Branch**
Switch to main branch:
git checkout main
git pull origin main

**Step 7: Switch Back to Feature Branch**
git checkout feature-update-readme

**Step 8: Merge Main into Feature Branch**
git merge main

**Step 9: Resolve Merge Conflicts**
If conflict occurs, Git shows:
CONFLICT (content): Merge conflict in README.md
Open the file:
<<<<<<< HEAD
Your change
=======
Main branch change
>>>>>>> main
Edit manually to keep correct content:
Combined final version of README content
Remove conflict markers.

**Step 10: Mark Conflict as Resolved**
git add README.md
git commit -m "Resolved merge conflict between main and feature branch"

**Step 11: Push Updated Feature Branch**
git push origin feature-update-readme

**Step 12: Create Pull Request**
1. Go to GitHub
2. Click **Compare & Pull Request**
3. Base: main
4. Compare: feature-update-readme
5. Click **Create Pull Request**

**Step 13: Lab Summary Format**
**Summary:**
- Created feature branch for README update.
- Modified README file.
- Pulled latest main branch.
- Merged main into feature branch.
- Encountered conflict in README file.
- Manually resolved conflict by combining changes.
- Committed resolved version.
- Pushed branch and created pull request.

**Sample Conflict Explanation**
Conflict occurred because both main branch and feature branch modified the same line in README.md. The conflict was resolved by manually editing the file and keeping the combined correct content.

**Result**
Thus, Git branching, merging, and merge conflict resolution were successfully performed.

| | | | |
|---|---|---|---|
| | Create a Static Website and Containerize, Build & Serve it using Docker. | | |

| | | | |
|---|---|---|---|
| **16** | Tasks:<br>1. Create a Simple Static Website (index.html file) with basic HTML content.<br>2. Write/create a Dockerfile to serve the website using Nginx.<br>3. Build the Docker Image<br>4. Run the container:<br>5. Access the Website using a Browser | Docker | CO3 |

**Aim**

To create a static website, containerize it using Docker with Nginx, build and run the Docker image, and access it through a browser using Git/GitHub for version control.

**Requirements**

- Git installed
- Docker installed
- GitHub account
- Browser

**Step 1: Create Project Folder**

mkdir static-website-docker
cd static-website-docker

- **Step 2: Initialize Git Repository**

git init

- **Step 3: Create Static Website**

Create file:
index.html
Add content:

```
<!DOCTYPE html>
<html>
<head>
   <title>My Static Website</title>
</head>
<body>
   <h1>Welcome to My Static Website</h1>
   <p>This website is served using Docker and Nginx.</p>
</body>
</html>
```

Save the file.

- **Step 4: Create Dockerfile**

Create file named **Dockerfile** (no extension):

```
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

- **Step 5: Check Files**

ls
Output:
Dockerfile
index.html

- **Step 6: Commit to Git**

git add .
git commit -m "Added static website and Dockerfile"

- **Step 7: Create GitHub Repository**
  1. Go to GitHub
  2. Click **New Repository**
  3. Name: static-website-docker
  4. Click **Create repository**

**Step 8: Push Code to GitHub**
git branch -M main
git remote add origin https://github.com/your-username/static-website-docker.git
git push -u origin main

**Step 9: Build Docker Image**
docker build -t static-website .

**Step 10: Run Docker Container**
docker run -d -p 8080:80 static-website

**Step 11: Verify Running Container**
docker ps

**Step 12: Access Website in Browser**
Open browser and go to:
http://localhost:8080
You will see:
Welcome to My Static Website
This website is served using Docker and Nginx.

**Screenshots to Capture for Lab**
1. GitHub repository page
2. Docker build output
3. Docker ps output
4. Website in browser

**Sample Lab Record Summary**
**Steps Performed:**
- Created static website using HTML.
- Created Dockerfile using Nginx image.
- Built Docker image successfully.
- Ran Docker container on port 8080.
- Website accessed through browser.
- Project uploaded to GitHub.

**Result**
Thus, a static website was successfully created, containerized using Docker, built, deployed using Nginx, and accessed through a browser.

| | | | |
|---|---|---|---|
| **17** | Create a Simple Python Flask API, Containerize the Application, Build & Push the Image using Docker and Deploy the Application using Kubernetes.<br><br>Tasks:<br>1. Create a Simple Flask API by writing a Python file (app.py) with basic endpoints.<br>2. Containerize the Flask App using Dockerfile.<br>3. Build the Image using Docker.<br>4. Push the Image to Docker Hub.<br>5. Create Kubernetes manifests (Deployment YAML & Service YAML) to deploy the application. | Docker & Kubernetes | CO3 |

| | | | |
|---|---|---|---|
| | 6. Apply the Manifests and Access the API via NodePort. | | |

**Aim**

To create a Flask API, containerize it using Docker, push the image to Docker Hub, and deploy the application using Kubernetes.

**Requirements**

- Python installed
- Docker installed
- Docker Hub account
- Kubernetes cluster (Minikube / Docker Desktop K8s)
- kubectl configured

**Step 1: Create Project Folder**

mkdir flask-k8s-app
cd flask-k8s-app

**Step 2: Create Flask API (app.py)**

Create file:
app.py
Add:

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return jsonify({"message": "Flask API is running!"})

@app.route("/hello")
def hello():
    return jsonify({"message": "Hello from Kubernetes Flask API"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

**Step 3: Create requirements.txt**

```
Flask
```

**Step 4: Create Dockerfile**

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

**Step 5: Build Docker Image**

Replace **yourusername** with Docker Hub username.
docker build -t yourusername/flask-k8s-api:1.0 .

**Step 6: Test Locally**
docker run -d -p 5000:5000 yourusername/flask-k8s-api:1.0
Open browser:
http://localhost:5000
http://localhost:5000/hello

**Step 7: Login to Docker Hub**
docker login

**Step 8: Push Image to Docker Hub**
docker push yourusername/flask-k8s-api:1.0

**Step 9: Create Kubernetes Deployment YAML**
Create file:
deployment.yaml
Content:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-api-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: flask-api
  template:
    metadata:
      labels:
        app: flask-api
    spec:
      containers:
      - name: flask-api
        image: yourusername/flask-k8s-api:1.0
        ports:
        - containerPort: 5000
```

**Step 10: Create Kubernetes Service YAML**
Create:
service.yaml
Content:

```
apiVersion: v1
kind: Service
metadata:
  name: flask-api-service
spec:
  type: NodePort
  selector:
    app: flask-api
  ports:
    - port: 5000
```

targetPort: 5000
nodePort: 30007

**Step 11: Apply Kubernetes Manifests**
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

**Step 12: Verify Pods and Service**
kubectl get pods
kubectl get svc

**Step 13: Access API from Browser**
If using Minikube:
minikube ip
Then open:
http://<minikube-ip>:30007
http://<minikube-ip>:30007/hello

**Screenshots to Capture for Lab**

1. Docker build success
2. Docker Hub repository
3. kubectl get pods
4. kubectl get svc
5. Browser output

**Sample Lab Record Summary**
**Steps Performed:**

- Created Flask API with two endpoints.
- Containerized using Dockerfile.
- Built and pushed image to Docker Hub.
- Created Kubernetes Deployment and Service YAML.
- Deployed application to Kubernetes.
- Accessed API using NodePort.

**Result**
Thus, the Flask API application was successfully containerized using Docker, pushed to Docker Hub, and deployed on Kubernetes using Deployment and Service.

| | | | |
|---|---|---|---|
| **18** | Set up a CI/CD pipeline to automate the building, testing, and deployment of a containerized application.<br><br>Tasks:<br>1. Set up Jenkins on a local machine or server.<br>2. Create a Dockerfile to containerize a sample application.<br>3. Write a Jenkinsfile to automate the process of building the Docker container, running tests, and deploying to a cloud platform (e.g., AWS or GCP).<br>4. Configure Jenkins to trigger builds upon code commits or pull requests. | Docker | CO4 |

**Aim**

To set up a CI/CD pipeline using Jenkins to automate building, testing, and deploying a Docker containerized application.

**Requirements**

- Docker installed
- Jenkins installed
- GitHub account
- Sample application (Python/Node/Java)
- Docker Hub account

**PART 1 — Install Jenkins**

**Step 1: Install Jenkins using Docker (Easy method)**

```
docker run -d -p 8080:8080 -p 50000:50000 --name jenkins \
-v jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

**Step 2: Get Jenkins Admin Password**

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Open browser:

http://localhost:8080

Paste password → Install suggested plugins → Create admin user.

**PART 2 — Create Sample Application**

**Step 3: Create project folder**

```
mkdir cicd-docker-app
cd cicd-docker-app
```

**Step 4: Create app.py**

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "CI/CD Pipeline with Jenkins and Docker"
```

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

**Step 5: Create requirements.txt**

```
Flask
```

**PART 3 — Create Dockerfile**

**Step 6: Dockerfile**

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python","app.py"]
```

**PART 4 — Push Code to GitHub**

```
git init
git add .
git commit -m "Initial CI/CD app"
git branch -M main
git remote add origin https://github.com/yourusername/cicd-docker-app.git
git push -u origin main
```

**PART 5 — Create Jenkinsfile**

**Step 7: Create Jenkinsfile**

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "yourusername/cicd-docker-app"
        TAG = "latest"
    }

    stages {

        stage('Clone Repo') {
            steps {
                git 'https://github.com/yourusername/cicd-docker-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:$TAG .'
            }
        }
```

```
        stage('Run Tests') {
            steps {
                echo "Running tests..."
                sh 'docker run --rm $IMAGE_NAME:$TAG python -c "print(\"Tests Passed\")"'
            }
        }

        stage('Push Image to Docker Hub') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'USER',
passwordVariable: 'PASS')]) {
                    sh 'echo $PASS | docker login -u $USER --password-stdin'
                    sh 'docker push $IMAGE_NAME:$TAG'
                }
            }
        }

        stage('Deploy Container') {
            steps {
                sh '''
                docker stop cicd-container || true
                docker rm cicd-container || true
                docker run -d -p 5000:5000 --name cicd-container $IMAGE_NAME:$TAG
                '''
            }
        }
    }
}
```

**PART 6 — Configure Jenkins Job**
**Step 8: Create Jenkins Pipeline Job**

1. Jenkins Dashboard → New Item
2. Name: CI-CD-Docker-Pipeline
3. Select **Pipeline** → OK

**Step 9: Configure Pipeline**
In pipeline section:

- Definition: **Pipeline script from SCM**
- SCM: Git
- Repo URL: GitHub repo URL
- Branch: main
- Script Path: Jenkinsfile

Save.

**PART 7 — Add Docker Hub Credentials**

1. Jenkins → Manage Jenkins → Credentials
2. Add Docker Hub username & password
3. ID: dockerhub

**PART 8 — Enable Auto Trigger**
**Step 10: Enable GitHub Webhook**
In GitHub repo:

Settings → Webhooks → Add webhook
Payload URL:
http://<jenkins-ip>:8080/github-webhook/
Content type: application/json
Enable push events.

**Step 11: Enable Jenkins Trigger**
In Jenkins job:
GitHub hook trigger for GITScm polling
Save.

**PART 9 — Trigger Pipeline**
Make any code change and push:
git commit -am "Pipeline trigger test"
git push
Jenkins pipeline runs automatically.

**PART 10 — Verify Deployment**
Open browser:
http://localhost:5000
Output:
CI/CD Pipeline with Jenkins and Docker

**Screenshots for Lab**

1. Jenkins pipeline success
2. Docker Hub image
3. Jenkins console output
4. Browser deployment output

**Lab Record Summary**
**Steps performed:**

- Installed Jenkins using Docker
- Created Dockerfile and application
- Created Jenkinsfile pipeline
- Configured Jenkins job
- Enabled webhook trigger
- Automated build, test, push, and deployment

**Result**
Thus, a CI/CD pipeline was successfully implemented using Jenkins and Docker to automate build, test, and deployment of a containerized application.

| | | | |
|---|---|---|---|
| **19** | Implement Continuous Deployment using GitHub Actions to deploy a Dockerized application.<br><br>Tasks:<br>1. Set up a GitHub repository and push a simple Dockerized application.<br>2. Create a GitHub Actions workflow to automatically build and push the Docker image to Docker Hub or GitHub Container Registry.<br>3. Automate deployment to a cloud platform (e.g., AWS ECS, Azure Kubernetes Service, or Google Kubernetes Engine).<br>4. Test the CI/CD pipeline by pushing new code changes and verifying that the deployment occurs automatically. | Docker | CO4 |

**Aim**

To implement Continuous Deployment using GitHub Actions to automatically build, push, and deploy a Dockerized application.

**Requirements**

- GitHub account
- Docker installed
- Docker Hub account
- Cloud platform (example: AWS EC2 / Azure VM / GCP VM) with Docker installed
- Basic Git knowledge

**PART 1 — Create Dockerized Application**

**Step 1: Create Project Folder**

```
mkdir github-actions-docker
cd github-actions-docker
```

**Step 2: Create app.py**

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Continuous Deployment using GitHub Actions"

if __name__ == "__main__":
```

```
app.run(host="0.0.0.0", port=5000)
```

**Step 3: Create requirements.txt**

```
Flask
```

**Step 4: Create Dockerfile**

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python","app.py"]
```

**PART 2 — Push Project to GitHub**

**Step 5: Initialize Git**

```
git init
git add .
git commit -m "Initial Dockerized Flask app"
```

**Step 6: Create GitHub Repository**

Name: github-actions-docker

**Step 7: Push to GitHub**

```
git branch -M main
git remote add origin https://github.com/yourusername/github-actions-docker.git
git push -u origin main
```

**PART 3 — Create GitHub Actions Workflow**

**Step 8: Create Workflow Folder**

mkdir -p .github/workflows

**Step 9: Create workflow file**

File name:

.github/workflows/docker-deploy.yml

**Step 10: Workflow Content**

```
name: Docker CI/CD Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build-push-deploy:

    runs-on: ubuntu-latest

    steps:

    - name: Checkout code
      uses: actions/checkout@v3

    - name: Login to Docker Hub
      uses: docker/login-action@v2
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}

    - name: Build Docker Image
      run: docker build -t ${{ secrets.DOCKER_USERNAME }}/github-actions-app:latest .

    - name: Push Docker Image
      run: docker push ${{ secrets.DOCKER_USERNAME }}/github-actions-app:latest

    - name: Deploy to Server via SSH
      uses: appleboy/ssh-action@v1.0.3
      with:
        host: ${{ secrets.SERVER_IP }}
        username: ${{ secrets.SERVER_USER }}
        key: ${{ secrets.SERVER_KEY }}
        script: |
```

```
        docker pull ${{ secrets.DOCKER_USERNAME }}/github-actions-app:latest
        docker stop github-actions-app || true
        docker rm github-actions-app || true
        docker run -d -p 5000:5000 --name github-actions-app ${{
secrets.DOCKER_USERNAME }}/github-actions-app:latest
```

**PART 4 — Configure GitHub Secrets**

Go to:

**GitHub Repo → Settings → Secrets → Actions → New Repository Secret**

Add:

| Secret Name | Value |
|---|---|
| DOCKER_USERNAME | Docker Hub username |
| DOCKER_PASSWORD | Docker Hub password |
| SERVER_IP | Cloud VM public IP |
| SERVER_USER | VM username |
| SERVER_KEY | Private SSH key |

**PART 5 — Prepare Cloud Server**

On AWS EC2 / Azure VM / GCP VM:

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo usermod -aG docker $USER
```

Restart session.

**PART 6 — Trigger Pipeline**

**Step 11: Make Code Change**

Edit app.py:

return "Auto deployed using GitHub Actions CI/CD"

**Step 12: Push Change**

```
git add .
git commit -m "Trigger GitHub Actions deployment"
git push
```

**PART 7 — Verify Pipeline**

1. Go to GitHub → Actions tab
2. Watch workflow run successfully
3. Docker image pushed to Docker Hub
4. Container redeployed automatically on server

**PART 8 — Access Application**

Open browser:

http://<SERVER_IP>:5000

Output:

Auto deployed using GitHub Actions CI/CD

**Screenshots for Lab**

1. GitHub Actions workflow success
2. Docker Hub image
3. Cloud VM running container
4. Browser output

**Lab Record Summary**

**Steps Performed:**

- Created Dockerized Flask application.
- Created GitHub Actions workflow.
- Automated Docker build and push.
- Automated deployment to cloud server.
- Verified Continuous Deployment by pushing new code.

**Result**

Thus, Continuous Deployment was successfully implemented using GitHub Actions and Docker.

| | | | |
|---|---|---|---|
| **20** | Create a GitHub Repository and Implement Version Control<br><br>Tasks:<br>• Set up a repository for a team project.<br>• Create branches for individual modules.<br>• Merge branches with pull requests after code reviews.<br>• Resolve conflicts during branch merging.<br>• Document the workflow using the README file.<br>• Submit the repository link | Git/GitHub | CO3 |

**Aim**

To create a GitHub repository, manage branches, merge using pull requests, resolve conflicts, and document the workflow using Git/GitHub.

**Requirements**

- Git installed
- GitHub account
- Internet connection

**Step 1: Create GitHub Repository**

1. Login to GitHub.
2. Click **New Repository**.
3. Enter:
    - Repository Name: team-project-version-control
    - Visibility: Public / Private
4. Click **Create Repository**.

**Step 2: Clone Repository**

```
git clone https://github.com/yourusername/team-project-version-control.git
cd team-project-version-control
```

**Step 3: Create README File**

```
touch README.md
```

Add:

```
# Team Project Version Control
```

This repository demonstrates GitHub version control workflow using branches and pull requests.

Commit:

```
git add README.md
git commit -m "Added README file"
git push
```

**Step 4: Create Branches for Modules**

```
git checkout -b module-auth
git checkout main
git checkout -b module-ui
git checkout main
git checkout -b module-api
git checkout main
```

**Step 5: Work on Module Branch**

Example for module-auth:

```
git checkout module-auth
touch auth.txt
echo "Authentication module code" > auth.txt
git add auth.txt
git commit -m "Added authentication module file"
git push origin module-auth
```

Repeat similar steps for other branches.

**Step 6: Create Pull Request**

1. Go to GitHub repo.
2. Click **Compare & Pull Request**.
3. Base: main
4. Compare: module-auth
5. Add title and description.
6. Click **Create Pull Request**.

**Step 7: Code Review**

Add review comment:

Code looks good. Ready to merge.

Approve pull request.

**Step 8: Merge Pull Request**

Click **Merge Pull Request → Confirm Merge**.

**Step 9: Simulate Conflict**

1. Edit README.md in main branch and commit.
2. Edit same line in module-ui branch and commit.
3. Create pull request from module-ui to main.
4. GitHub shows conflict.

**Step 10: Resolve Conflict**

1. Click **Resolve conflicts**.
2. Manually edit content.
3. Remove conflict markers.
4. Mark as resolved and commit.

**Step 11: Update README Workflow Documentation**

Edit README.md:

## Workflow

1. Create feature branches for modules.
2. Commit changes to respective branches.
3. Create pull requests.
4. Review and approve changes.
5. Resolve conflicts manually if any.
6. Merge into main branch.

Commit and push.

**Screenshots for Lab**

1. Repository page

2. Branch list
3. Pull request
4. Conflict resolution screen
5. README workflow section

**Sample Lab Record Summary**

**Steps Performed:**

- Created GitHub repository.
- Created module branches.
- Added code to branches.
- Created pull requests.
- Reviewed and merged branches.
- Resolved merge conflicts.
- Documented workflow in README.

**Result**

Thus, GitHub repository and version control workflow was successfully implemented using Git and GitHub.